

2024 - 1º fecha

8.

a. Especificar un programa que calcule el mcd , es decir el máximo común divisor de dos números enteros. Comentario: en la postcondición se puede usar la notación $mcd(x, y)$.

Asumo que por especificación solo se pide dar la terna de Hoare $\{p\}S\{q\}$ sin necesidad de definir explícitamente S .

Una terna posible podría ser $\{x = X \wedge y = Y\}S\{z = mcd(X, Y)\}$

b. Justificar por qué $(x > 0, y = x)$ no es una especificación correcta de un programa que debe devolver, en la variable de salida y , el valor de la variable de entrada x .

La especificación no es correcta porque no asegura que la variable y tenga el valor que tenía x al iniciar el programa, ya que x podría haber sido modificado luego de haber iniciado el programa. Se deberían usar variables lógicas.

c. El hecho de que un programa satisfaga la especificación $(x > 0, x > 0)$, ¿asegura que la aserción $x > 0$ se cumpla a lo largo de toda su ejecución? Justificar la respuesta.

No, no asegura la aserción. Podría pasar que $x = 8$ al inicio del programa, luego se ejecute la instrucción $x = -5$ y finalmente, antes de terminar el programa, se ejecute la instrucción $x = 2$. Si bien se cumple que $x > 0$ tanto antes de iniciar el programa como después de terminada su ejecución, no se cumplió a lo largo de TODA su ejecución.

9. Asumiendo que se cumple la fórmula de correctitud total $\langle p \rangle S \langle q \rangle$ indicar, justificando la respuesta, si se cumple cada uno de los siguientes enunciados:

a. Si S no termina, entonces su estado inicial no satisface p .

Por contrapositiva, el enunciado es equivalente a decir: si el estado inicial satisface p , entonces S termina. Esto es **verdadero** por definición de correctitud total: si p se cumple entonces necesariamente S termina en un estado que satisface q .

b. Si S termina en un estado que satisface q , entonces su estado inicial satisface p

•

Nuevamente, por contrapositiva, el enunciado es equivalente a decir: si el estado inicial no satisface p , entonces S no termina, o termina en un estado que no satisface q . Esto es **falso**, puede ocurrir que S termine en un estado que sí satisface q , o incluso que nunca termine. Como ya la precondition no se cumple, todo lo demás es irrelevante.

c. Si S termina en un estado que no satisface q , entonces su estado inicial no satisface p .

Contrapositiva: si el estado inicial satisface p , entonces S no termina, o termina en un estado que satisface q . Esto necesariamente es **verdadero**, ya que el consecuente posee una disyunción que solo es falsa cuando todas sus condiciones son falsas, y si analizamos lo que tenemos, es imposible que esto suceda. Si S no termina, claramente no termina en un estado que satisface q . Si S termina en un estado que satisface q , claramente sí terminó. Por lo tanto, suceda lo que suceda, la implicación es verdadera.

10. Probar mediante el método H , usando el invariante p , la fórmula de correctitud parcial $\{p\}\text{while } \neg q \text{ do skip od}\{q\}$.

1. Elementos:

i. Precondición: p

ii. Postcondición: q

iii. Condición del while: $\neg q$

iv. Invariante: p

2. Probar que la precondition implica al invariante antes de entrar al while:

i. $p \rightarrow p$

ii. Se cumple trivialmente, toda variable proposicional se implica a sí misma.

3. Probar que el invariante se preserva en cada iteración:

i. $\{invariante \wedge condición\}cuerpo\{invariante\}$

ii. $\{p \wedge \neg q\}skip\{p\}$

iii. Se cumple trivialmente ya que $(p \wedge \neg q) \rightarrow p$ es una tautología.

4. Probar la salida del while:

i. $invariante \wedge \neg condición \rightarrow postcondición$

ii. $(p \wedge \neg(\neg q)) \rightarrow q$

iii. $(p \wedge q) \rightarrow q$

iv. Lo anterior se cumple por ser una tautología: la única forma de que la fórmula sea falsa es que el antecedente sea V y el consecuente F . Pero si $q = F$, que es la única forma de que el consecuente sea falso, entonces el antecedente también es falso por tener un AND lógico. Por lo tanto la fórmula se cumple siempre.

2025 - Ejemplo de examen

5. La sintaxis de un determinado lenguaje de programación L se define de la siguiente manera. Un programa S pertenece a L si y sólo si S tiene la forma:

$$S :: skip \mid x := e \mid S_1; S_2 \mid \text{case of } B_1 \rightarrow S_1; B_2 \rightarrow S_2; \text{otherwise} \rightarrow S_3 \text{ end}$$
$$e :: n \mid x \mid e_1 + e_2$$
$$B ::= true \mid false \mid e_1 = e_2 \mid \neg B$$

donde e es una expresión entera, n es una constante entera, x es una variable entera, y B es una expresión booleana. Sea $var(S)$ la expresión que denota el conjunto de las variables que aparecen en el programa S . Definir por inducción el conjunto $var(S)$.

- **Expresiones enteras:**

- $var(n) = \emptyset$ porque n no es una variable.
- $var(x) = \{x\}$ porque x es una variable.
- $var(e_1 + e_2) = var(e_1) \cup var(e_2)$

- **Expresiones booleanas:**

- $var(true) = \emptyset$
- $var(false) = \emptyset$
- $var(e_1 = e_2) = var(e_1) \cup var(e_2)$
- $var(\neg B) = var(B)$

- **Programas:**

- $var(skip) = \emptyset$
- $var(x := e) = \{x\} \cup var(e)$
- $var(S_1; S_2) = var(S_1) \cup var(S_2)$
- $var(\text{case of } B_1 \rightarrow S_1; B_2 \rightarrow S_2; \text{otherwise} \rightarrow S_3 \text{ end}) = var(B_1) \cup var(S_1) \cup var(B_2) \cup var(S_2) \cup var(S_3)$

6. Probar empleando el método de verificación H :

a. $\{p\}\text{while } true \text{ do skip od}\{q\}$

1. Se elige como invariante a p .
2. $\{p \wedge true\}skip\{p\}$
3. $\{p\}skip\{p\}$ (una variable y verdadero siempre es equivalente a la variable)
4. Lo anterior es válido por la regla del skip.
5. $\{p\}\text{while } true \text{ do skip od}\{p \wedge \neg true\}$ (aplicación de la regla del while)
6. $\{p\}\text{while } true \text{ do skip od}\{p \wedge false\}$
7. $\{p\}\text{while } true \text{ do skip od}\{false\}$
8. Dado que $false \Rightarrow q$, por regla de consecuencia reemplazamos $false$ por q .
9. $\{p\}\text{while } true \text{ do skip od}\{q\}$

b. $\{p\}\text{while } \neg q \text{ do skip od}\{q\}$

Ya resuelto en el parcial anterior.

2025 - 1º fecha

8. Responder:

a. ¿Cuándo un programa es correcto parcialmente con respecto a una especificación?

Un programa es correcto **parcialmente** con respecto a una especificación cuando dada una precondition y una postcondición, si se cumple la precondition y el programa termina entonces la postcondición se cumple.

b. ¿Cuándo un programa es correcto totalmente con respecto a una especificación?

Un programa es correcto **totalmente** con respecto a una especificación cuando dada una precondition y una postcondición, si se cumple la precondition entonces el programa termina y la postcondición se cumple.

c. ¿Por qué se hace la distinción entre correctitud parcial y correctitud total?

La distinción se hace primordialmente para diferenciar a los programas que siempre terminan (totalmente correctos) vs los que **pueden** no terminar (parcialmente correctos).

9. Especificar un programa que calcule la raíz cuadrada de un número x que sea entero y mayor estricto que cero, y tal que al final, x tenga el mismo valor que al inicio del programa.

$$\{x = X \wedge x \in \mathbb{Z} \wedge x > 0\} S \{y = \sqrt{X} \wedge x = X\}$$

10. Probar con el método H :

a. $\{x = X \wedge X > 0\} y := 2x \{x = X \wedge y \geq 2\}$.

1. $\{x = X \wedge X > 0\} y := 2x \{x = X \wedge y \geq 2\}$
2. $\{x = X \wedge 2x \geq 2\} y := 2x \{x = X \wedge y \geq 2\}$ (ASI)
3. $(x = X \wedge X > 0) \Rightarrow (x = X \wedge 2x \geq 2)$?
4. $(x = X \wedge X > 0) \Rightarrow (x > 0) \Rightarrow (x \geq 1) \Rightarrow (2x \geq 2) \Rightarrow (x = X \wedge 2x \geq 2)$
5. Por lo tanto, por regla CONS, la precondition original $x = X \wedge X > 0$ implica a la obtenida $(x = X \wedge 2x \geq 2)$ y por ende la terna es válida.

b. $\{true\} \text{while true do skip od} \{false\}$

1. Tomamos como invariante a la misma precondition: $i = true$
2. $\{true \wedge true\} \text{skip} \{true\}$ (REP)
3. $\{true\} \text{skip} \{true\}$ (lógica)
4. $\{true\} \text{while true do skip od} \{true \wedge \neg true\}$ (REP)
5. $\{true\} \text{while true do skip od} \{true \wedge false\}$ (lógica)
6. $\{true\} \text{while true do skip od} \{false\}$ (lógica)

Desconocido

8. Indicar si las siguientes fórmulas de correctitud son verdaderas o falsas. Justificar la respuesta en cada caso. Comentario: no hay que utilizar los métodos H y H^* , sólo aplicar las definiciones de

correctitud parcial y total, y recordar que las variables son de tipo entero:

a. $\{true\}x := x + y\{x \geq y\}$

La precondition $true$ se satisface en todo estado y el programa termina siempre porque es una simple asignación. Sin embargo, tomando el estado inicial σ tal que $\sigma(x) = -2$ y $\sigma(y) = 0$, luego de ejecutar $x := x + y$ se obtiene $x = -2$. La postcondición $x \geq y$ resulta falsa, ya que $-2 \not\geq 0$.

Como existe un estado que satisface la precondition para el cual el programa termina pero la postcondición no se cumple, entonces la fórmula de correctitud es **falsa**.

b. $\{x = 0\} \text{while } y = 0 \text{ do } z := z + 1 \text{ od } \{x = 0\}$

Está claro que si $y = 0$ al inicio del programa, el programa no termina.

Si tomamos el estado σ tal que $\sigma(x) = 0$ y $\sigma(y) = 0$ tenemos que la precondition se cumple pero el programa no termina por lo tanto nunca se llega a evaluar la postcondición.

Si tomamos el estado σ tal que $\sigma(x) = 0$ y $\sigma(y) \neq 0$ tenemos que la precondition se cumple pero el programa termina sin hacer nada por no cumplirse la condición del bucle, y además se cumple la postcondición.

Por lo tanto en cualquier caso donde se cumpla la precondition y el programa termine se cumple la postcondición, por lo tanto la fórmula de correctitud es **verdadera**.

c. $\{x > 0 \wedge y > 0\} \text{while } y > x \text{ do } y := y - x \text{ od } \{true\}$

La única forma de que una fórmula de correctitud parcial sea falsa es que exista un estado inicial que satisfaga la precondition, para el cual el programa termine y la postcondición no se cumpla. Como la postcondición es $true$, ésta se satisface **para todo estado final**. Por lo tanto, dicho contraejemplo no puede existir y la fórmula de correctitud es **verdadera**.

9. Dado un programa S del lenguaje PLW estudiado en clase, justificar en cada caso por qué son verdaderos los siguientes enunciados:

a. Que exista algún estado σ tal que $\sigma \models p$ y $val(\pi(S, \sigma)) \models q$ no implica que $\models \{p\}S\{q\}$

La implicación no se cumple porque estamos hablando de cosas distintas. En el primer caso solo exigimos que haya UN estado σ tal que se cumple la precondition p y al ejecutar S se llega a un estado final que cumple q . En el segundo caso

requerimos que para TODO estado σ tal que se cumple la precondition p , al ejecutar S se llegue a un estado final que cumpla q .

b. Que exista algún estado σ tal que $\sigma \not\models p$ y $val(\pi(S, \sigma)) \not\models q$ no implica que $\not\models \langle p \rangle S \langle q \rangle$

Por contrarrecíproca: $\models \langle p \rangle S \langle q \rangle$ no implica que no pueda existir algún estado σ tal que $\sigma \not\models p$ y $val(\pi(S, \sigma)) \not\models q$.
Contraejemplo:

$\langle x = 0 \rangle skip \langle x = 0 \rangle$

1. Se cumple $\models \langle p \rangle S \langle q \rangle$ por definición.
2. Sin embargo, existe un estado σ tal que $\sigma(x) = 1$ de forma tal que tanto la precondition como la postcondition, luego de ejecutar el programa, son ambas falsas.

10. Probar con el método H la fórmula $\{p\} \text{while } p \text{ do } S \text{ od } \{q\}$, asumiendo que en H se prueba la fórmula $\{p\} S \{p\}$. Ayuda: Hay que probar con H la formula $\{p\} \text{while } p \text{ do } S \text{ od } \{q\}$ partiendo de $\{p\} S \{p\}$. Tener en cuenta: (a) a partir de $\{p\} S \{p\}$ se puede probar $\{p \wedge p\} S \{p\}$ (por la regla de consecuencia CONS), desde donde entonces luego se puede aplicar la regla de la repetición REP. (b) La aserción false implica cualquier otra.

1. $\{p\} S \{p\}$ (hipótesis)
2. $\{p \wedge p\} S \{p\}$ (por CONS en 1. ya que $p \Rightarrow p \wedge p$)
3. $\{p\} \text{while } p \text{ do } S \text{ od } \{p \wedge \neg p\}$ (aplicación de regla REP sobre 2.)
4. $\{p\} \text{while } p \text{ do } S \text{ od } \{false\}$ (por CONS en 3. ya que $(p \wedge \neg p) \Rightarrow false$)
5. $\{p\} \text{while } p \text{ do } S \text{ od } \{q\}$ (por CONS en 4. ya que como la aserción false implica cualquier otra, tenemos $false \Rightarrow q$)